Registro de Decisiones de Arquitectura (ADR)

Plataforma de Aprendizaje Gamificado — Lote 2 (ADR-0004 a ADR-0006)

Versión 1 — OFICIAL

Fecha del documento: 22 de septiembre de 2026

ADR-0004: Estrategia de desacople entre módulos

Fecha: 22 de septiembre de 2026

Estado: Aceptada

Contexto

El proyecto adoptó un monolito modular (ADR-0002). Se necesita definir reglas concretas para que los módulos permanezcan realmente desacoplados entre sí, evitando que el monolito degrade con el tiempo hacia un sistema entrelazado y difícil de mantener, dado que el equipo es de 1-2 personas y debe poder modificar un módulo sin temor a romper otros.

Opciones consideradas

- Sin reglas explícitas de desacople: cada módulo accede a lo que necesite, según convenga en el momento.
- Desacople mediante capas por módulo (dominio/aplicación/infraestructura), fachadas con DTOs, eventos internos e inyección de dependencias.
- Desacople estricto tipo microservicios, pero manteniendo un solo despliegue (mayor sobre-ingeniería).

Decisión

Se adoptan cinco reglas de desacople para todos los módulos: (1) arquitectura en capas — dominio, aplicación e infraestructura — dentro de cada módulo; (2) ningún módulo accede directamente a las tablas de otro módulo; (3) cada módulo expone una fachada pública con contratos claros, usando DTOs y nunca sus modelos internos; (4) comunicación por eventos internos (event emitter) para casos que no requieren respuesta inmediata; (5) inyección de dependencias para las implementaciones concretas (ej. acceso a base de datos).

Trade-offs

Aspecto	Análisis
A favor	Permite modificar un módulo sin romper otros; facilita pruebas al poder inyectar dependencias falsas; deja preparado el camino para separar un módulo en servicio independiente en el futuro, si fuera necesario.
En contra / riesgos	Añade algo de disciplina y código adicional (fachadas, DTOs, eventos) comparado con simplemente llamar funciones o tablas directamente; requiere mantener esta disciplina de forma constante para que no se erosione con el tiempo.
Alternativa descartada	No tener reglas explícitas es más rápido al inicio, pero con alta probabilidad de que el monolito se vuelva frágil y entrelazado a medida que crezca, dificultando el mantenimiento por una sola persona.

ADR-0005: Adopción de lenguaje ubicuo y glosario del dominio

Fecha: 22 de septiembre de 2026

Estado: Aceptada

Contexto

El proyecto maneja conceptos propios del negocio (XP, gemas, desafíos, ranking, pensum, skins) que deben nombrarse de forma consistente en documentación y código, para evitar ambigüedad y facilitar el mantenimiento por parte de un equipo pequeño.

Opciones consideradas

- Usar nombres genéricos y técnicos en el código (ej. points, coins, score), sin relación directa con el negocio.
- Definir un lenguaje ubicuo: un glosario oficial de términos del dominio, reflejado tanto en la documentación como en el código.

Decisión

Se adopta el lenguaje ubicuo del proyecto y se crea un glosario oficial (versión inicial de prueba) con los términos del dominio (Estudiante, Pensum, Desafío, XP, Gema, Skin, Ranking, Suscripción, entre otros), incluyendo una sección de términos a evitar. Este glosario se usará de forma consistente en nombres de funciones, variables y tablas.

Trade-offs

Aspecto	Análisis
A favor	El código queda alineado con el negocio, facilitando que cualquier persona (incluido un futuro colaborador) entienda rápidamente qué hace cada parte; reduce ambigüedad entre módulos (ej. "nivel" de experiencia vs. de dificultad).
En contra / riesgos	Requiere mantener el glosario actualizado a medida que el dominio evoluciona; si no se revisa, el código y la documentación pueden desalinearse con el tiempo.
Alternativa descartada	Usar nombres genéricos es más rápido de escribir al inicio, pero dificulta el mantenimiento y la comunicación sobre el negocio a mediano plazo.

ADR-0006: Definición de Bounded Contexts

Fecha: 22 de septiembre de 2026

Estado: Aceptada

Contexto

Cada módulo del monolito (autenticación, suscripciones, contenido, desafíos, gamificación, administración) modela al mismo tiempo conceptos compartidos del negocio, como el "Estudiante", pero desde perspectivas distintas. Es necesario definir límites explícitos para que cada módulo tenga un modelo y lenguaje propio, evitando una entidad única y frágil que todo el sistema dependa de ella.

Opciones consideradas

- Un único modelo compartido de "Usuario/Estudiante" usado por todos los módulos.
- Bounded Contexts: cada módulo modela solo lo que le interesa del concepto compartido, con su propio lenguaje y límites explícitos.

Decisión

Se adopta la definición de Bounded Contexts: cada módulo es un límite explícito de modelo y lenguaje. Por ejemplo, Autenticación modela al Estudiante como una identidad con credenciales; Gamificación lo modela como un jugador con XP y ranking; Suscripciones lo modela como un cliente con plan y estado de pago. Se elaborará un mapa de contextos que muestre cómo se comunican entre sí (vía fachadas o eventos, según lo definido en el ADR-0004).

Trade-offs

Aspecto	Análisis
A favor	Evita una entidad "Usuario" gigante y frágil compartida por todo el sistema; refuerza el desacople ya definido; cada módulo puede evolucionar su modelo sin afectar a los demás.
En contra / riesgos	Implica cierta duplicación de datos básicos del Estudiante entre contextos (ej. su identificador se repite en varios módulos), lo cual es intencional en DDD pero puede parecer redundante a primera vista.
Alternativa descartada	Un modelo único de "Usuario" es más simple de entender al inicio, pero se vuelve un punto de fragilidad: cualquier cambio en él afecta potencialmente a todos los módulos.